

CS 630: Software Testing and Quality Assurance

3-1 Journal: Evaluating a Test Plan Against QA Standards

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

July 21, 2026

Evaluating a Sample Test Plan

The StrongLifts 5x5 sample test plan demonstrates several effective quality assurance practices while also revealing areas where additional documentation and planning would improve overall software quality. A well-developed test plan should provide sufficient guidance to testers, define the scope and objectives of testing, identify risks, and establish measurable criteria for determining whether software is ready for release. According to IEEE 29119, software testing documentation should support consistency, traceability, and communication throughout the testing process (IEEE, 2021). Likewise, the Context-Driven School emphasizes that testing should be adapted to the needs of the project and the risks involved rather than following a rigid process (Kaner et al., 2002). The sample plan follows several of these principles but also contains notable omissions that could reduce testing effectiveness.

One of the strengths of the sample test plan is its clear definition of testing scope. The document states that the purpose of the plan is to support pre-release regression testing for the StrongLifts mobile application before deployment to Android and iOS users. It also explains why regression testing is especially important due to delays in application store updates, meaning users could experience defects for several days after release. This demonstrates an understanding of business risk and aligns with context-driven testing by adapting the testing strategy to the realities of mobile application deployment.

Another strength is the organization of testing into feature-based sessions. Instead of relying on detailed scripted test cases, the plan divides the application into manageable testing sessions covering features such as workout programs, history, cloud synchronization, subscriptions,

Apple Health integration, and complete user journeys. This approach supports exploratory testing while ensuring major functional areas to receive attention. The inclusion of regression sessions for complex workflows, such as syncing data across multiple devices, reflects good risk-based planning because these scenarios are likely to expose defects that could significantly affect users.

The plan also includes useful metrics that allow the team to evaluate testing progress. Tracking the number of testing sessions completed, defects discovered, active showstopper defects, and total testing duration provides management with measurable information about software readiness. These metrics support informed release decisions and encourage continuous monitoring of software quality.

Although the plan contains several strengths, it overlooks important elements recommended by IEEE 29119 documentation standards. One significant omission is the lack of clearly defined entry and exit criteria. The document specifies testing duration and scheduling expectations but never explains the specific conditions that must be satisfied before testing begins or before a release is approved. Without objective completion criteria, different team members may reach different conclusions regarding whether testing is complete, increasing the possibility of releasing software with unresolved issues.

A second weakness is the absence of detailed risk assessment. While the plan prioritizes testing sessions numerically, it does not explain why certain features receive higher priority or identify the potential business impact if specific functionality fails. IEEE documentation recommends documenting testing risks and their associated mitigation strategies to ensure that the most

critical areas receive appropriate attention (IEEE, 2021). Including a formal risk analysis would improve resource allocation and provide stronger justification for testing priorities.

Another important omission is traceability between requirements and testing activities. The sample plan lists features and user journeys but does not connect them to documented software requirements or acceptance criteria. As a result, there is no simple way to verify that every requirement has been tested completely. Requirement traceability is essential because it helps ensure complete test coverage while making it easier to identify gaps when new functionality is added or existing features change.

The plan also provides very little information about the testing environment. It mentions Android and iPhone platforms but does not specify supported operating system versions, device models, browser compatibility where applicable, network conditions, or hardware configurations. Mobile applications often behave differently depending on the device or operating system version. Without documenting these environments, testers may unintentionally overlook defects that only occur under specific conditions.

Furthermore, the document focuses almost entirely on functional regression testing and gives minimal attention to nonfunctional quality attributes. There are no testing activities addressing performance, usability, accessibility, security, reliability, or compatibility. Modern mobile applications frequently process personal information and synchronize data across cloud services, making security and performance testing particularly important. Failure to evaluate these quality characteristics could result in software that functions correctly but still delivers a poor user experience or exposes sensitive user information.

These omissions could negatively affect software quality and stakeholder confidence. Missing entry and exit criteria create uncertainty regarding release of readiness, while limited risk documentation increases the chance that high-impact defects remain undiscovered. The absence of requirement traceability makes it difficult to demonstrate complete test coverage, which could reduce confidence among project managers and clients. Likewise, failing to document testing environments or include nonfunctional testing increases the likelihood that users encounter unexpected failures after deployment, resulting in additional maintenance costs and reduced customer satisfaction.

Several practical revisions would improve the completeness of the test plan. First, the document should include clearly defined entry criteria, such as completion of development, successful unit testing, and deployment of a stable build, along with exit criteria based on measurable standards like defect severity, test completion percentage, and stakeholder approval. Second, adding a formal risk assessment matrix would explain testing priorities and help focus on limited testing resources on the highest-risk functionality. Third, creating a requirements traceability matrix would ensure that every software requirement is linked to corresponding testing activities and expected results. The plan should also document supported devices, operating systems, network configurations, and testing tools to improve repeatability. Finally, expanding the scope to include performance, security, usability, and accessibility testing would provide more comprehensive quality assurance coverage.

Reviewing this sample test plan has strengthened my understanding of effective QA planning. While I appreciate the flexibility of session-based exploratory testing, I also recognize that successful testing requires sufficient documentation to ensure consistency and accountability. In my own future projects, I plan to develop test plans that combine risk-based testing with clear

documentation, measurable success criteria, and complete traceability to software requirements. I also intend to include both functional and nonfunctional testing activities to improve overall software quality and increase stakeholder confidence before release. Applying IEEE 29119 guidance together with context-driven testing principles will help me produce testing strategies that are both structured and adaptable to the specific needs of each project.

References

IEEE. (2021). *IEEE Standard for Software and Systems Test Documentation (IEEE 29119-4)*.

IEEE Standards Association.

Kaner, C., Bach, J., & Pettichord, B. (2002). *Lessons learned in software testing: A context-driven approach*. John Wiley & Sons.